111703040 TsungYen Yu

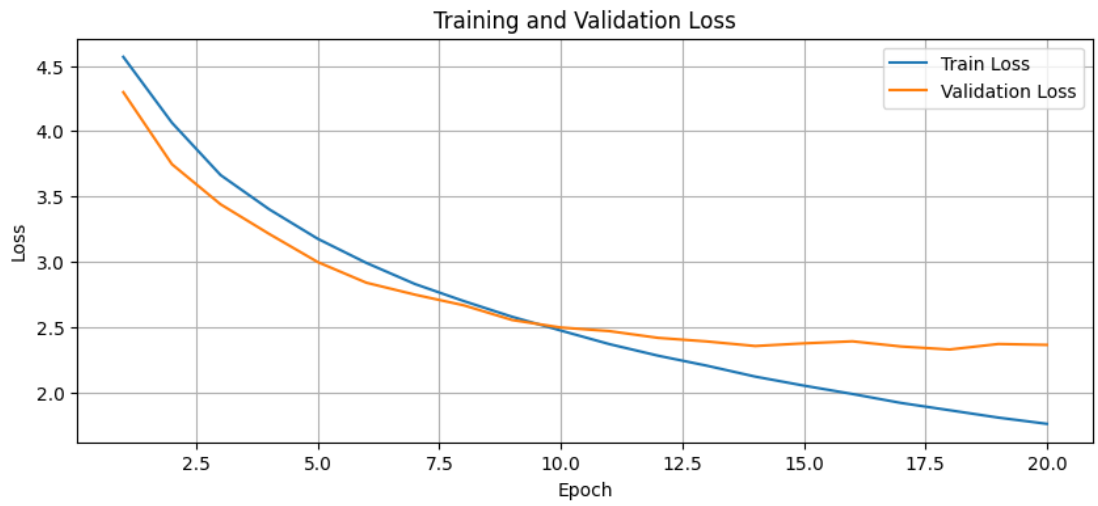
1. Baseline CNN: Make It Deeper (30%)

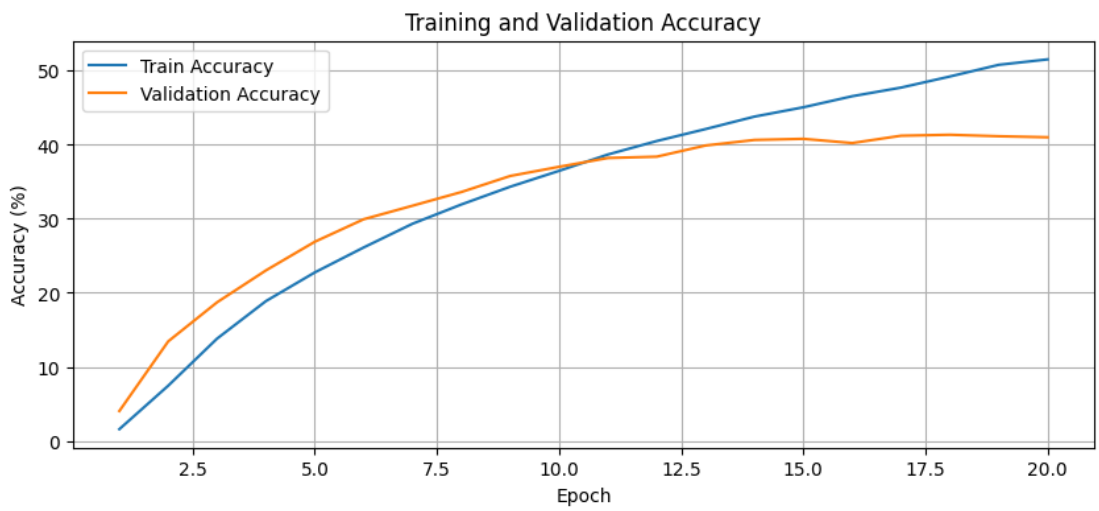
1. Complete the baseline CNN model code to have 3 convolutional layers based on the comments in the provided code block, we have already defined the structure of each block. *(Hint: Each max pooling reduces the spatial size by half — make sure your tensor shapes still match in the forward() function!)* (10%)

```
[8]
0
秒
from torchsummary import summary
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
summary(model,(3,32,32))
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 16, 32, 32]             448
         MaxPool2d-2           [-1, 16, 16, 16]               0
            Conv2d-3           [-1, 32, 16, 16]           4,640
         MaxPool2d-4             [-1, 32, 8, 8]               0
            Conv2d-5             [-1, 64, 8, 8]          18,496
         MaxPool2d-6             [-1, 64, 4, 4]               0
           Dropout-7                 [-1, 1024]               0
            Linear-8                  [-1, 500]         512,500
           Dropout-9                  [-1, 500]               0
           Linear-10                  [-1, 100]          50,100
================================================================
Total params: 586,184
Trainable params: 586,184
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 0.29
Params size (MB): 2.24
Estimated Total Size (MB): 2.54
----------------------------------------------------------------
```

2. Complete baseline training code and Train the original CNN until convergence (5%)





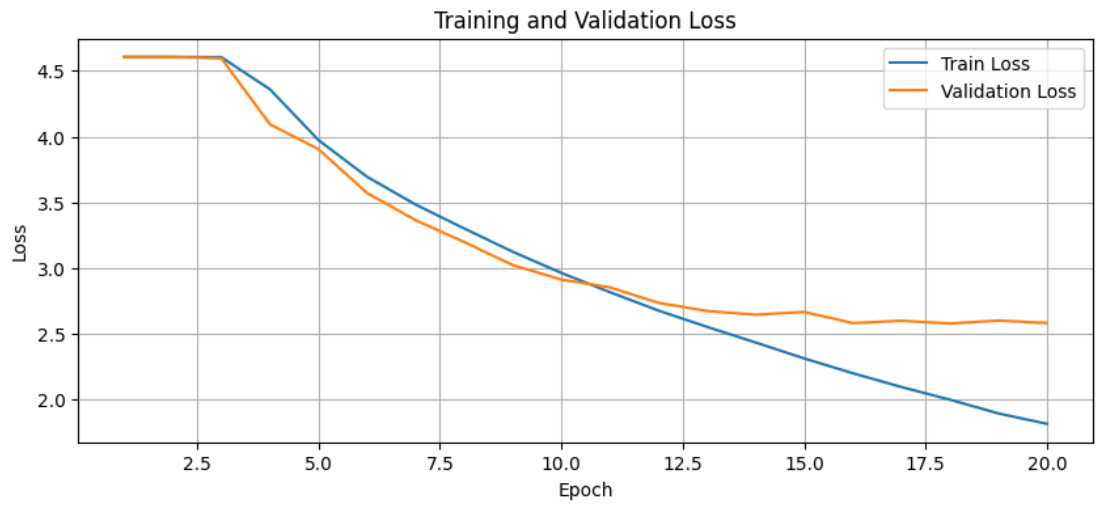
Since the Validation Acc stops improving at epoch 15 ~ 20, I conclude that it converges.

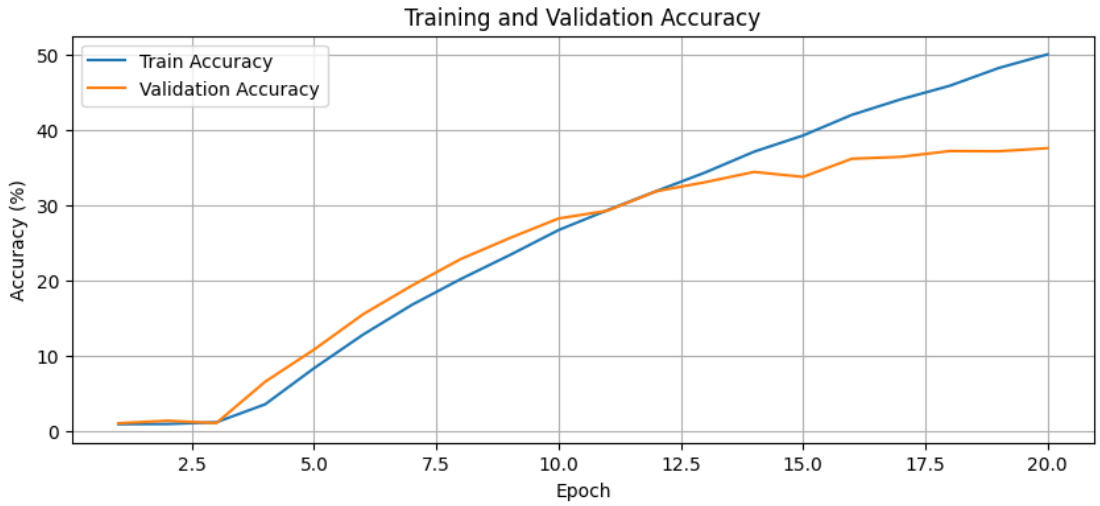
3. Extend the baseline CNN from 3 convolutional layers to 5 layers. You can decide the number of channels for each layer. (5%)

```
[32]
0
秒

from torchsummary import summary
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
summary(model,(3,32,32))

----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1          [-1, 16, 32, 32]             448
         MaxPool2d-2          [-1, 16, 16, 16]               0
            Conv2d-3          [-1, 32, 16, 16]           4,640
         MaxPool2d-4            [-1, 32, 8, 8]               0
            Conv2d-5            [-1, 64, 8, 8]          18,496
         MaxPool2d-6            [-1, 64, 4, 4]               0
            Conv2d-7            [-1, 96, 4, 4]          55,392
            Conv2d-8           [-1, 128, 4, 4]         110,720
           Dropout-9                 [-1, 2048]               0
           Linear-10                  [-1, 500]       1,024,500
          Dropout-11                  [-1, 500]               0
           Linear-12                  [-1, 100]          50,100
================================================================
Total params: 1,264,296
Trainable params: 1,264,296
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 0.32
Params size (MB): 4.82
Estimated Total Size (MB): 5.16
----------------------------------------------------------------
```

I added a 96-channel convolutional layer and a 128-channel convolutional layer both without pooling (since 4 by 4 is already small).

4. Train the modified CNN until convergence and compare it with the original simple CNN, adjust the number of epochs if needed. (5%)





Although the Validation Acc is slightly moving up, the Loss has already frozen at 2.5. Therefore, I conclude that it converges at epoch around 20.

Notice that at the beginning of the training, there's a horizontal line. Because there's no such phenomenon in the baseline model, we can tell that it's not the learning rate's fault. It must be the 2 added convolutional layers.

5. Which one is better, and why? (5%)

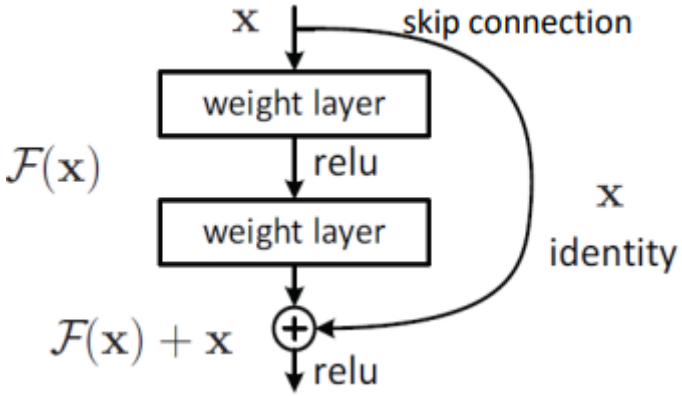
   In this case, the baseline is better, because the network is not too shallow and not too deep. On the other hand, the latter model is too deep so that we encounter Vanishing Gradients Effect. That's why there's an obvious flat line at the beginning and the accuracy is lower.

2. Residual Block: To Be Even Deeper (15%)

1. What is a residual block? In your own words, briefly explain what it is

and why it was introduced. *(Hint: Think about vanishing gradients and training stability.)* (5%)

Since initially all the weights are small, when the network becomes deeper, the gradient becomes smaller as the backpropagation has to multiply lots of small numbers. If, somehow, we can find a way to avoid getting values under 1 when taking derivatives, the problem is solved. That's what a residual block serves.



With an additional x added to the output, the derivatives will be at least 1 because the derivative of x itself is 1.

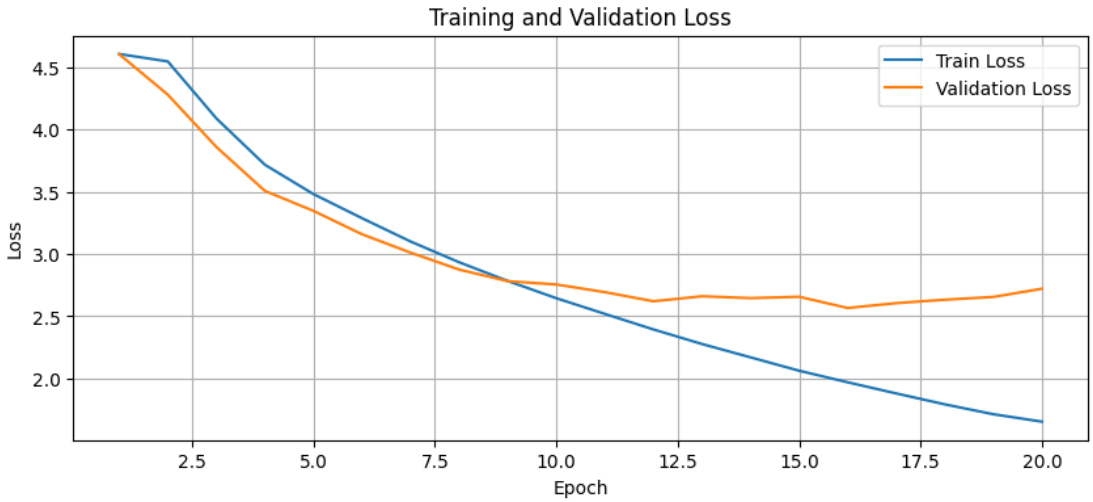
2. Modify your improved CNN from Step 1. Replace two of your convolutional layers with residual modules. (You can either define a custom ResidualBlock class or directly implement the skip connections inside your forward() function.) (5%)

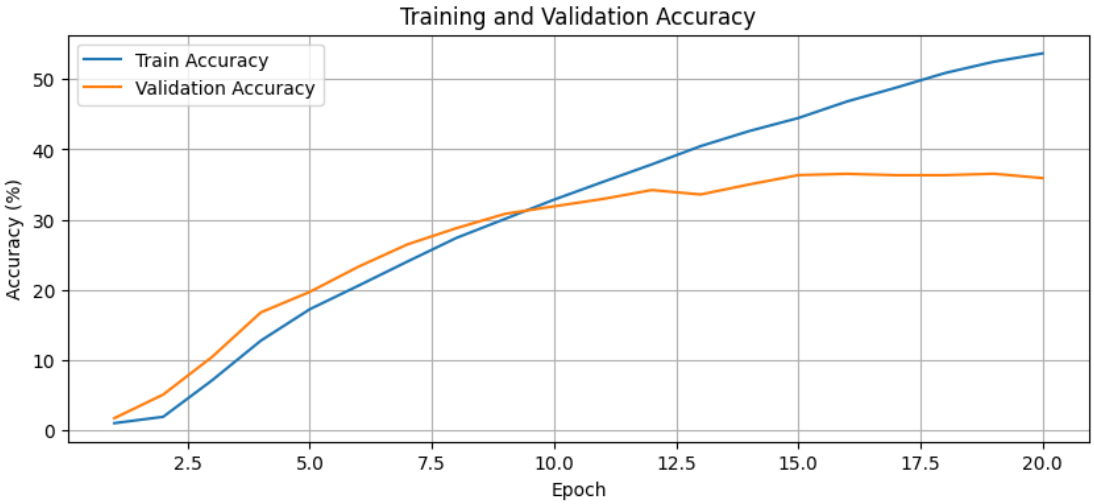
```
[38]
from torchsummary import summary
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
summary(model,(3,32,32))
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 16, 32, 32]             448
         MaxPool2d-2           [-1, 16, 16, 16]               0
            Conv2d-3           [-1, 32, 16, 16]           4,640
         MaxPool2d-4             [-1, 32, 8, 8]               0
            Conv2d-5             [-1, 64, 8, 8]          18,496
         MaxPool2d-6             [-1, 64, 4, 4]               0
            Conv2d-7             [-1, 96, 4, 4]          55,392
            Conv2d-8             [-1, 96, 4, 4]           6,240
     ResidualBlock-9             [-1, 96, 4, 4]               0
           Conv2d-10            [-1, 128, 4, 4]         110,720
           Conv2d-11            [-1, 128, 4, 4]          12,416
    ResidualBlock-12            [-1, 128, 4, 4]               0
          Dropout-13                 [-1, 2048]               0
           Linear-14                  [-1, 500]       1,024,500
          Dropout-15                  [-1, 500]               0
           Linear-16                  [-1, 100]          50,100
================================================================
Total params: 1,282,952
Trainable params: 1,282,952
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 0.38
Params size (MB): 4.89
Estimated Total Size (MB): 5.29
----------------------------------------------------------------
```

Because my custom layers have different number of channels (96 and 128) with the default number 3 layer (64), we need to apply point wise convolution (1 by 1 Conv2D) to make it addable. That's why in each ResBlock there's 2 Conv2D (the main 3 by 3 conv and a PWC).

3. Train your new residual CNN and compare. Evaluate the model and compare the results with previous models, and answer: Which one performs better, and why do you think that is? (5%)

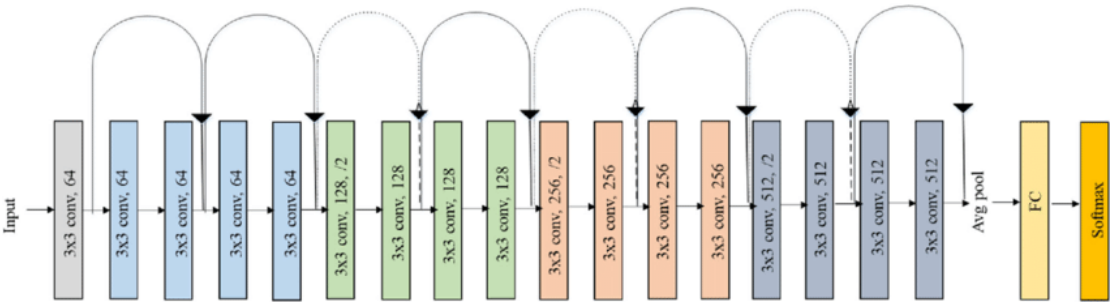




At the end, Baseline > ResBlock > Modified-5-layer. With the added 2 ResBlock, there's no longer obvious flat line at the beginning of training. However, the Validation Loss and Acc are still worse than the baseline model.

3. Classic Architecture: ResNet-18 from Scratch (15%)

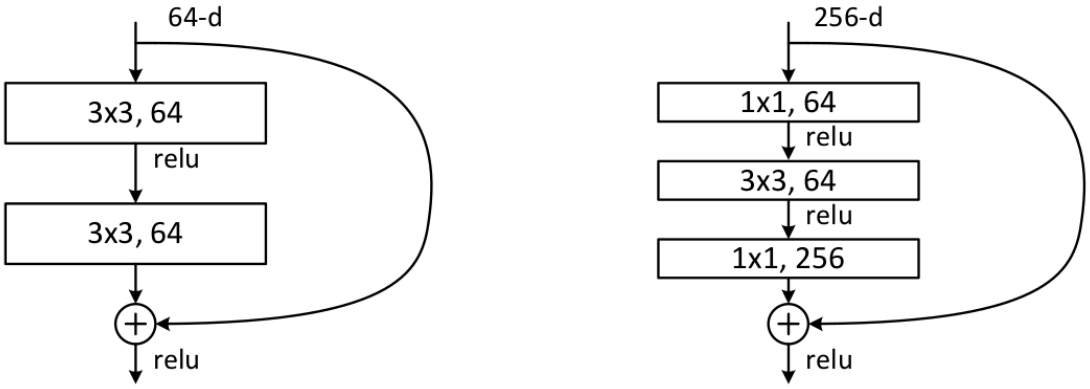
1. What is ResNet-18? In your own words, explain what ResNet-18 is and why it's a well-known architecture. (5%)



ResNet-18 is an 18-layer CNN that has Skip Connections every 2 layers. It's well-known because it's relatively shallow compared to the other ResNet family and it's pretty accurate as a baseline model.

2. Compare ResNet-18, 34, and 50. What's the difference between these versions in terms of architecture, depth, and use cases? (5%)

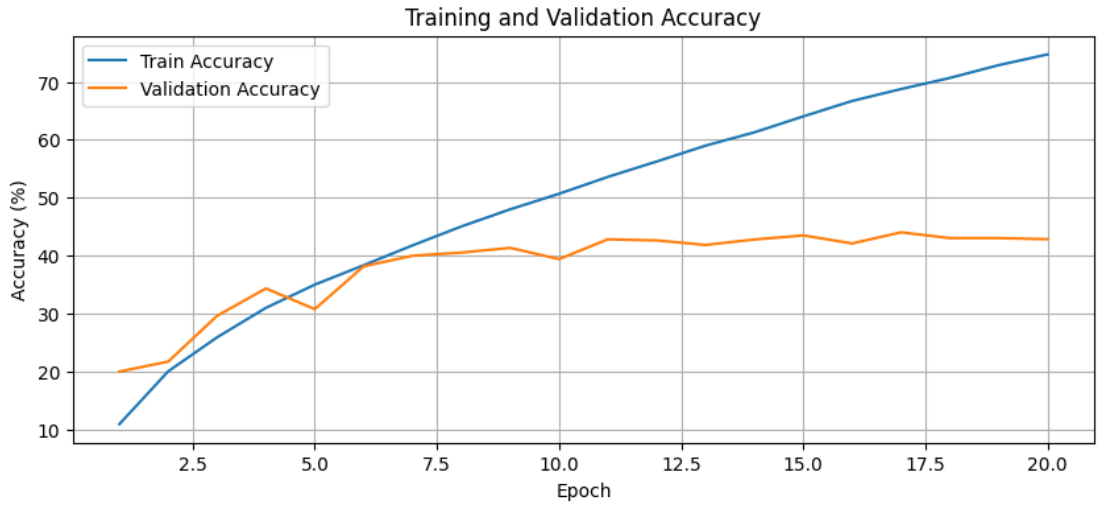
   There are 2 major differences, first, ResNet-18 has 18 layers, ResNet-34 has 34 layers, and ResNet-50 has 50 layers. Secondly, since ResNet-50 is too deep to put lots of 3 by 3 convolutional kernels, there is a little alternation:



   For ResNet-50, instead of straightly apply a Skip Connection, we first apply point wise convolution, to save the number of parameters.

3. Train ResNet-18 from scratch (no pretrained weights).

   Use torchvision.models.resnet18(pretrained=False) to load the model, modify the final layer to fit 100 classes, and train it on CIFAR-100. Compare the result with your previous CNNs — how does it perform? (5%)



Surprisingly, the Validation Acc is still horrible. However, the Training Loss and Acc are a lot better than the baseline model. That's a good sign, it means the model is learning at a better speed.

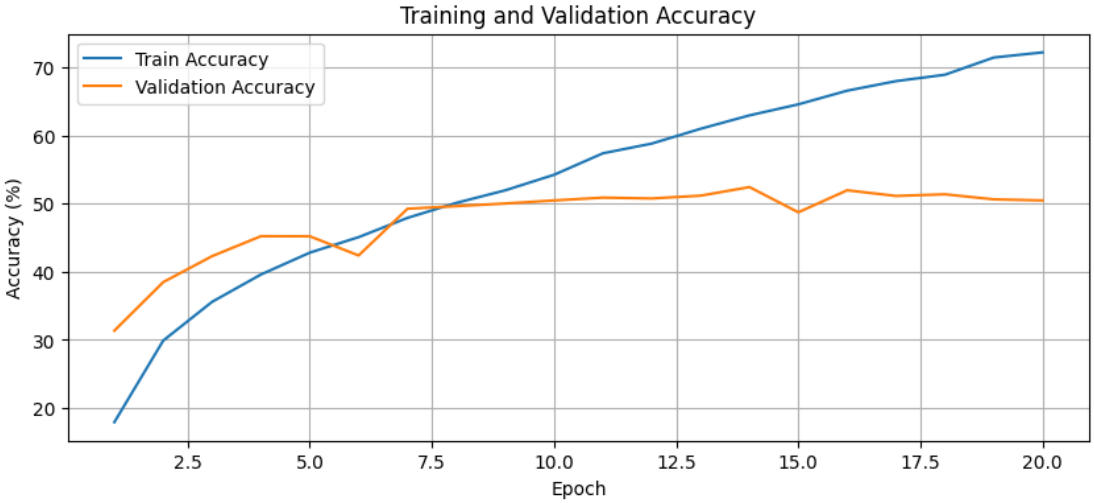
4. **Transfer Learning: Using Pretrained ResNet-18 (20%)**

   1. What is transfer learning, and what problem is it trying to solve? Briefly explain the core idea behind transfer learning and why it is useful in deep learning. (5%)

      Transfer learning is like building LEGOs on top of other finished ones. We reuse a pre-trained model and train it so that it can solve a different but similar problem. The reason behind this technique is that it's too exhausting to train a model from scratch every time. We have to gather data, label them, and find a powerful GPU to train it with tremendous amounts of time. With transfer learning, since the early layers of a well-known network learn universal features that are

applicable to many different tasks, we reuse some of the layers from it to extract features for our own task.

2. Load and fine-tune a pretrained ResNet-18.
   Use models.resnet18(pretrained=True) and replace the final layer to match 100 classes, to train the model on CIFAR-100 and record the results. (5%)




```
# key in the current model
trained_model = resnet18_pretrain.to(device)

trained_model.load_state_dict(torch.load('best-model.pt'))
criterion = nn.CrossEntropyLoss().to(device)
test_loss, test_acc = evaluate(trained_model, test_iterator, criterion, device)

print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')
```

```
Test Loss: 1.927 | Test Acc: 51.17%
```

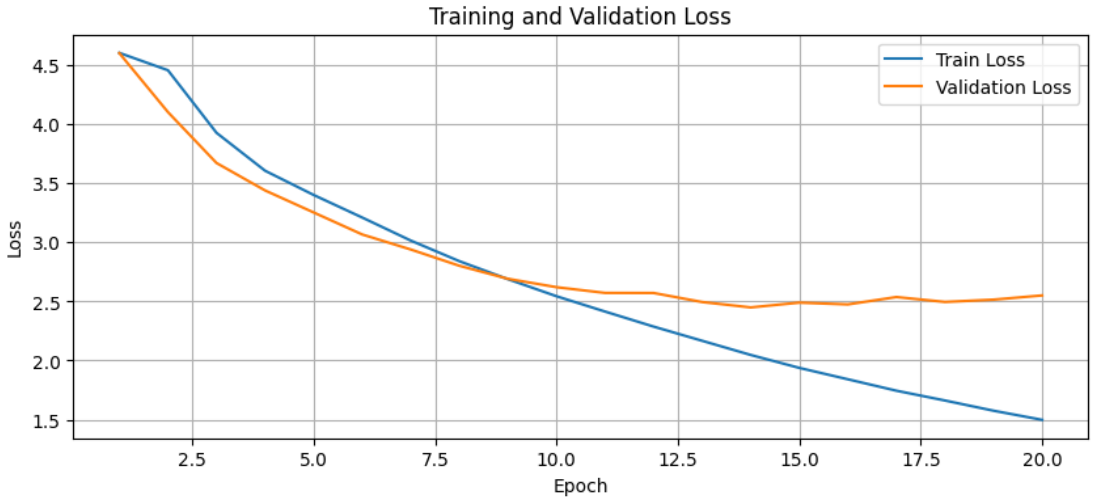
3. Compare the performance. How does the pretrained ResNet-18 compare to all models in the previous steps. (5%)

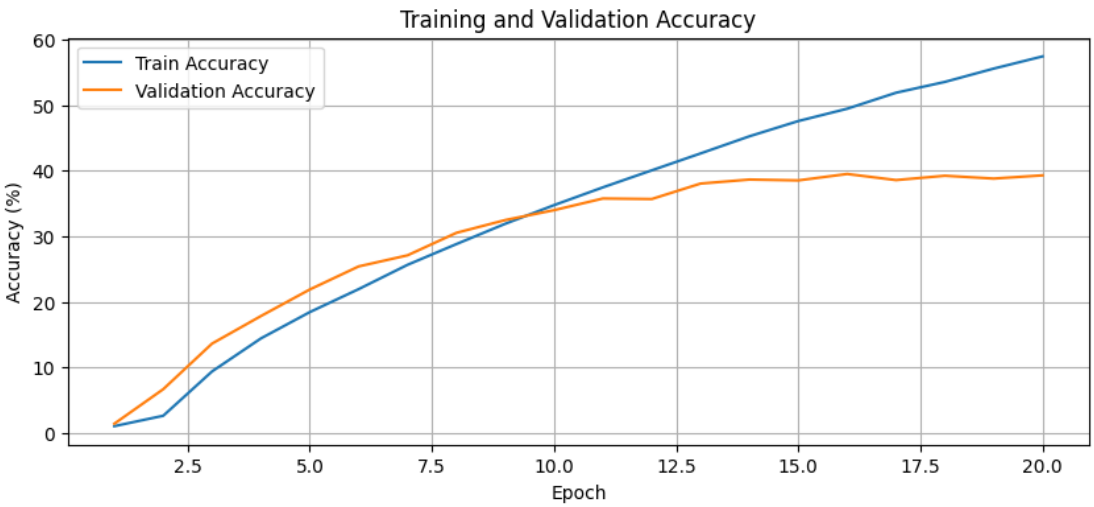
Obviously, the Validation Acc improves a lot, and the model stops learning relatively early (around 10 epochs). The reason is primarily the pre-trained parameters. That's why the model converge quickly because there's only few parameters are not yet trained.

4. Is transfer learning always better? Share your thoughts: What are the potential risks or limitations of transfer learning?
   (e.g., domain mismatch, overfitting to small data, feature mismatch, etc.) (5%)
   Although transfer learning often perform pretty well, there are some unignorable risks and limitations. For example, when the source domain and the target domain are too dissimilar, it's hard for a good pre-train network to help. Overfitting to target data is also a common one, but we can combine the original training data with our target ones to mitigate this issue (so that the model don't forget the original one and cause overfitting).

5. **More Ways of Classic Deep Learning Modules (20%)**
   1. Inception Module. Replace the residual blocks from Step 2 with an Inception-style design. For example, you might apply 1x1, 3x3, and 5x5 convolutions in parallel and concatenate their outputs. (10%)

```
[8]
from torchsummary import summary
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
incep_model = incep_model.to(device)
summary(incep_model,(3,32,32))

----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 16, 32, 32]             448
         MaxPool2d-2           [-1, 16, 16, 16]               0
            Conv2d-3           [-1, 32, 16, 16]           4,640
         MaxPool2d-4             [-1, 32, 8, 8]               0
            Conv2d-5             [-1, 64, 8, 8]          18,496
         MaxPool2d-6             [-1, 64, 4, 4]               0
            Conv2d-7             [-1, 42, 4, 4]           2,730
            Conv2d-8             [-1, 42, 4, 4]          24,234
            Conv2d-9             [-1, 44, 4, 4]          70,444
   InceptionBlock-10            [-1, 128, 4, 4]               0
          Dropout-11                 [-1, 2048]               0
           Linear-12                  [-1, 500]       1,024,500
          Dropout-13                  [-1, 500]               0
           Linear-14                  [-1, 100]          50,100
================================================================
Total params: 1,195,592
Trainable params: 1,195,592
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 0.33
Params size (MB): 4.56
Estimated Total Size (MB): 4.90
----------------------------------------------------------------
```



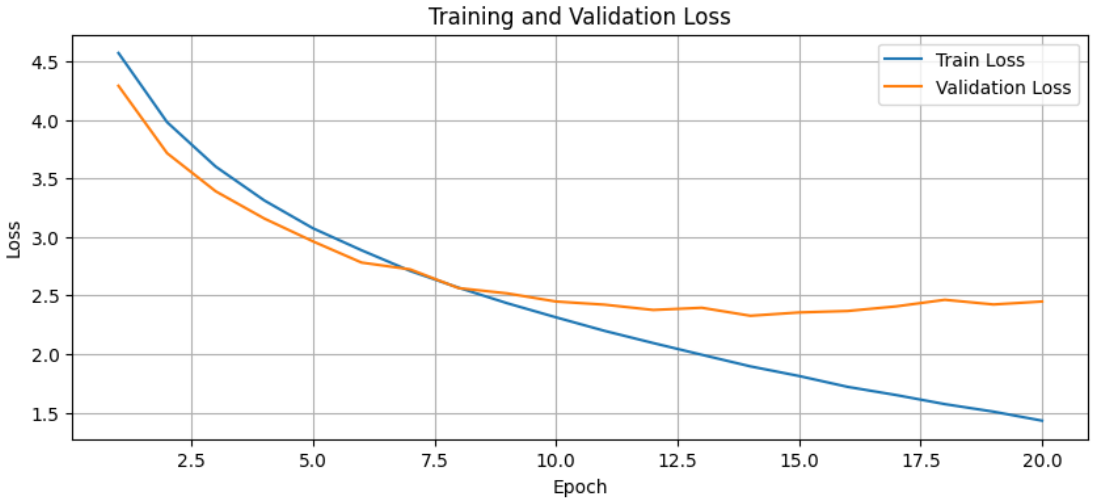


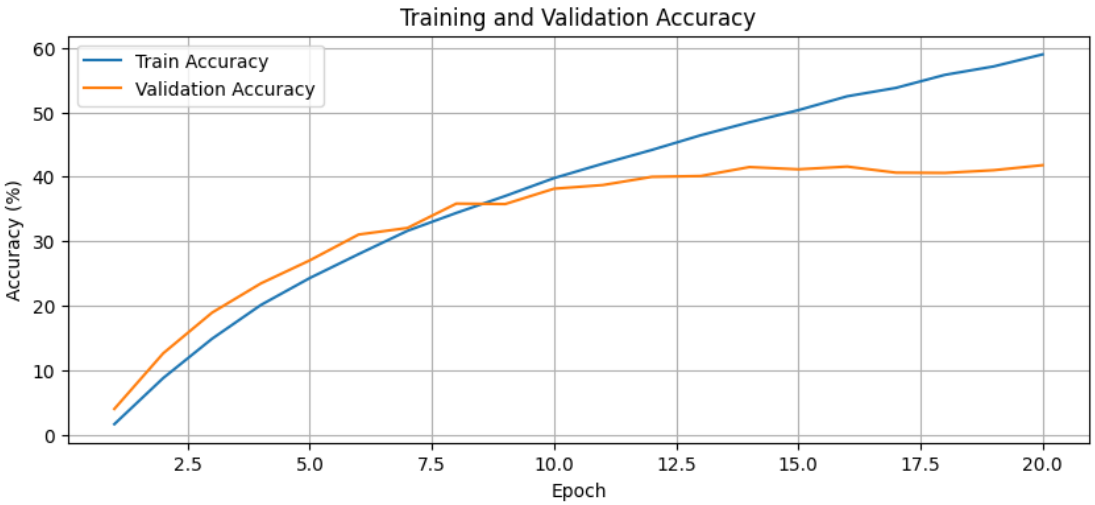
Since we already know that there is Gradient Vanishing problem when we have 5 convolutional layers without residual mechanism, I straightly replace the 2 Residual Blocks with only 1 Inception Block. It turns out that there is no obvious flat line at the beginning and the Validation Acc is also better than the original 5 convolutional layer model (It's also better than the Residual Block model too).

2. DenseNet-style Module. Try implementing a DenseNet-style block, where each layer receives the concatenated outputs from all previous layers in the block. This encourages feature reuse and improves gradient flow. You can use torch.cat() to concatenate feature maps inside the block. (10%)

```python
from torchsummary import summary
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
dense_model = dense_model.to(device)
summary(dense_model,(3,32,32))
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 16, 32, 32]             448
         MaxPool2d-2           [-1, 16, 16, 16]               0
            Conv2d-3           [-1, 32, 16, 16]           4,640
         MaxPool2d-4             [-1, 32, 8, 8]               0
            Conv2d-5             [-1, 64, 8, 8]          18,496
         MaxPool2d-6             [-1, 64, 4, 4]               0
            Conv2d-7             [-1, 32, 4, 4]          18,464
        DenseBlock-8             [-1, 96, 4, 4]               0
            Conv2d-9             [-1, 32, 4, 4]          27,680
       DenseBlock-10            [-1, 128, 4, 4]               0
          Dropout-11                 [-1, 2048]               0
           Linear-12                  [-1, 500]       1,024,500
          Dropout-13                  [-1, 500]               0
           Linear-14                  [-1, 100]          50,100
================================================================
Total params: 1,144,328
Trainable params: 1,144,328
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 0.33
Params size (MB): 4.37
Estimated Total Size (MB): 4.71
----------------------------------------------------------------
```





We can see that there are no Gradient Vanishing problems (just like ResBlock) and the Validation Acc is better than both ResBlock model and InceptionBlock model (but still lose to the baseline model).